Phase 3 Report: Reasoning Finetuning and Post-Finetune Attention Analysis

Building and Analyzing Monolingual Transformer LMs in Two Indian Languages

Erva Rishitha (2025201027)

September 16, 2026

Abstract

Phase 3 finetunes the two pretrained models from Phase 2 – Model H (Telugu) and Model L (Nepali) – on their own synthetic comparative-reasoning datasets, written from scratch in each language's own script. Both go from 0.00% exact-match accuracy to 64.69% (Telugu) and 52.74% (Nepali) on a held-out test set that includes a relation pattern never seen during training. Most of the work in this phase went into two problems: a first configuration that learned the reasoning task while degrading general-language perplexity by 264%, and a sampling imbalance that kept the numeric comparison templates stuck in single digits. This report covers the dataset, both fixes, the results with error analysis, and a pretrained-vs-finetuned attention comparison.

1 Synthetic Reasoning Dataset

Each language has its own programmatically generated corpus – nothing downloaded, and no data, entity names, or output shared between the two generators. Four template families:

- **pairwise** – two entities, one numeric attribute (age, height, price, weight), which is greater or smaller. Example (Telugu): "వాణికి 12 సంవత్సరాలు వయస్సు ఉంది. రాజేష్కి 70 సంవత్సరాలు వయస్సు ఉంది. వీరిలో ఎవరు వయసులో చిన్నవారు?" ("Vani is 12, Rajesh is 70. Which is younger?")
- **threeway** – three entities, one attribute, asking for the maximum or minimum.
- **transitive** – two chained inequalities with a question about the endpoints. Contains *no numbers at all*: the answer requires chaining the two stated relations.
- **transitive_4hop** – a three-hop chain over four entities, generated **test-only**. The model never sees a chain longer than two statements during training, so accuracy here measures whether it learned the relation or just the sentence shape.

I chose this set because the templates fail for different reasons. Pairwise and threeway need the model to recover numeric magnitude from tokenized digits; transitive needs no numeric ability at all but does need entity tracking across clauses. When accuracy diverges between them, the gap points at a specific capability rather than at general difficulty.

Entity names are split into disjoint train/val/test pools *before* any example is generated, so no name seen during finetuning reappears at evaluation. Each template also carries several statement and question phrasings chosen at random, so the task cannot be solved by memorising one fixed shape.

Table 1: Reasoning dataset sizes. `transitive_4hop` is test-only by construction.

Language	Split	pairwise	threeway	transitive	4hop	Total
Telugu	train	1,500	1,500	1,499	0	4,499
Telugu	val	250	250	233	0	733
Telugu	test	250	250	248	399	1,147
Nepali	train	1,500	1,500	1,497	0	4,497
Nepali	val	250	250	235	0	735
Nepali	test	250	250	250	399	1,149

2 Finetuning Setup

Each model starts from its own Phase 2 checkpoint (step 30,500), keeps its own tokenizer and 12,000-token vocabulary fixed, and never sees the other language's data or weights. Examples are tokenized as `[BOS]+prompt+answer+[EOS]` and padded to 128 tokens, with every label position before the answer span masked to `ignore_index` – so all gradient signal comes from predicting the answer, never from reproducing the prompt. Checkpoints use the same resume-capable schema as pretraining.

2.1 Catastrophic forgetting

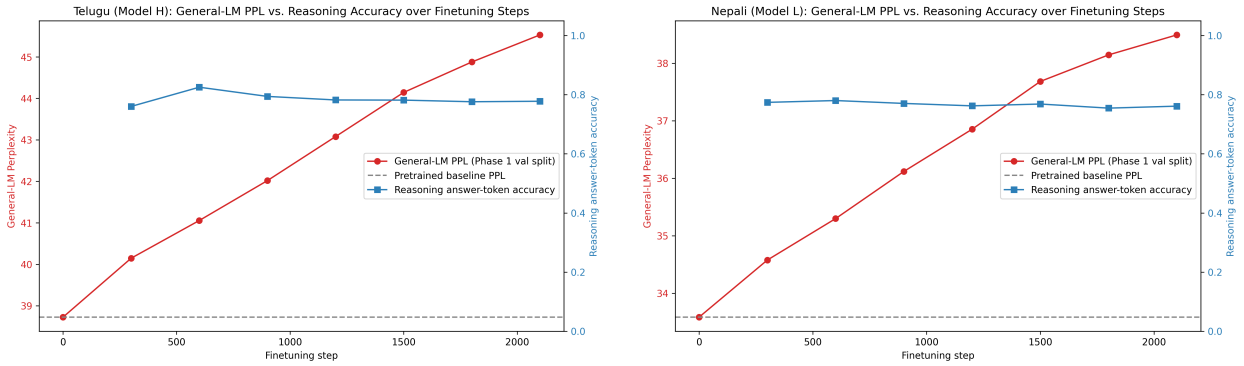
My first configuration looked fine on the reasoning metric while quietly destroying the model as a language model: perplexity on the Phase 1 validation split (ordinary text, unrelated to the task) rose from 38.7 to 140.9, an increase of about 264%. Reasoning accuracy could not detect this, because the two are measured on disjoint distributions – the reasoning task's outputs are drawn from a tiny closed set of entity names, so a model can get very good at emitting one of two names while getting far worse at everything else in the language. From then on I measured general-LM perplexity at every checkpoint alongside accuracy.

Three changes fixed it:

- **Learning rate** $3 \times 10^{-5} \rightarrow 1 \times 10^{-5}$**.** The most important one. Large updates on a 4,500-example dataset with a narrow output distribution move the pretrained weights substantially within a few hundred steps.
- **Dropout** $0.1 \rightarrow 0.15$**, then reverted.** An attempt at extra regularization that made things worse: validation accuracy rose, peaked, then *declined*, which is not the shape of a healthy regularized run. Learning a new task through noisier activations than the model was pretrained with cost more than it bought.
- **Early stopping on validation accuracy, not loss.** Loss kept rising even where accuracy was flat. That is a confidence artifact – cross-entropy punishes insufficiently peaked probability mass – not a capability one, so I selected checkpoints on accuracy.

Table 2: General-LM perplexity vs. reasoning accuracy across finetuning checkpoints. Step 0 is the Phase 2 pretrained checkpoint.

	Telugu (Model H)			Nepali (Model L)		
Step	Gen. PPL	Δ%	Reas. acc.	Gen. PPL	Δ%	Reas. acc.
0 (pretrained)	38.73	–	–	33.58	–	–
300	40.15	+3.7%	76.0%	34.58	+3.0%	77.4%
600 (chosen)	**41.06**	**+6.0%**	**82.5%**	**35.30**	**+5.1%**	**78.0%**
900	42.02	+8.5%	79.4%	36.12	+7.5%	77.0%
1,200	43.08	+11.2%	78.2%	36.85	+9.7%	76.2%
1,500	44.14	+14.0%	78.2%	37.68	+12.2%	76.8%
2,100	45.53	+17.6%	77.8%	38.49	+14.6%	76.1%



(a) Telugu (Model H)

(b) Nepali (Model L)

Figure 1: Reasoning accuracy peaks at step 600 and flattens, while general-LM perplexity degrades monotonically. The gap between the two curves is the argument for stopping early.

In both languages accuracy peaks at step 600 and never improves after, while perplexity degrades monotonically and never recovers. No later step buys reasoning ability at an acceptable price, so step 600 is the clear choice: +6.0% general-LM degradation for Telugu and +5.1% for Nepali, against the +264% my first attempt reached.

2.2 Curriculum template rebalancing

Uniform sampling over the three training templates fails here because of a difficulty imbalance, not a count imbalance. `transitive` saturates almost immediately, so most of the gradient budget goes to re-teaching a solved pattern. Across every uniform-sampling run, Telugu `pairwise` accuracy sat between 4% and 12% and would not respond to a lower learning rate, more data, or longer training – which was the clue that the problem was in the sampling distribution rather than in optimization.

The fix is to weight the sampler 3:1 toward the easy template early, interpolating linearly to 3:1 toward the hard templates by the end. Once `transitive` is solved its examples contribute almost no loss and therefore almost no gradient, so uniform sampling wastes most of each batch; shifting the mixture concentrates signal where the model is still wrong. Telugu `pairwise` went from the 4–12% band to **46.00%**, and `transitive` did not regress (99.60% final).

Table 3: Final finetuning hyperparameters (identical for both languages)

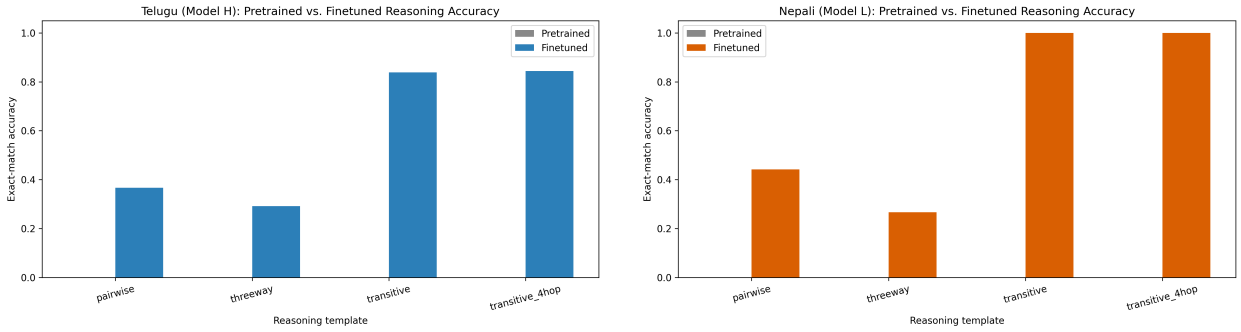
Hyperparameter	Value
Starting checkpoint	That language's own Phase 2 step-30,500 checkpoint
Tokenizer / vocabulary	Fixed from Phase 1 (12,000, per-language)
Sequence length (finetune)	128 (positional table remains 512)
Learning rate	1×10^{-5}, 150-step warmup + cosine decay
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$), fresh state
Weight decay / dropout / grad clip	0.01 / 0.1 / 1.0
Batch size	16
Max steps	3,000, early stopping on val accuracy (patience 12)
Loss masking	Answer span only (prompt $\rightarrow$ `ignore_index`)
Curriculum weights	easy 3.0 $\rightarrow$ 1.0, hard 1.0 $\rightarrow$ 3.0
Selected checkpoint	Step 600 (both languages)

3 Results

Evaluation is greedy decoding of the answer span followed by strict exact match after whitespace stripping – no partial credit, no other normalization.

Table 4: Pretrained vs. finetuned exact-match accuracy on the held-out test set

	Telugu (Model H)		Nepali (Model L)	
Template	Pretrained	Finetuned	Pretrained	Finetuned
pairwise	0.00%	46.00%	0.00%	32.80%
threeway	0.00%	21.20%	0.00%	11.60%
transitive	0.00%	99.60%	0.00%	88.40%
transitive_4hop (held-out)	0.00%	81.95%	0.00%	68.67%
Overall	**0.00%**	**64.69%**	**0.00%**	**52.74%**



(a) Telugu (Model H)

(b) Nepali (Model L)

Figure 2: Exact-match accuracy by template, before and after finetuning.

Why 0.00% before finetuning is expected. The pretrained models are next-token predictors over news and web text that have never seen "సమాధానం:" ("Answer:") as an instruction

to emit one entity and stop, so they continue the passage instead. Given a weight-comparison prompt, pretrained Telugu produces "1. బరువును తగ్గించడానికి, బరువును తగ్గించడానికి..." ("1. To reduce weight, to reduce weight..."), and pretrained Nepali produces "- यो एक उचाइ हो।" ("- This is a height."). These are fluent continuations, just not answers, so exact match necessarily scores zero. The figure measures instruction-format compliance, not reasoning ability.

3.1 Error analysis

The real result in Table 4 is the ordering: the two templates containing *no numbers* score far above the two requiring numeric comparison, in both languages.

Numbers do not tokenize with place-value structure. BPE segments numbers by corpus frequency rather than digit position, so the segmentation is inconsistent from number to number – Telugu encodes 146 as `[_14][6]` but 198 as a single token `[_198]`, and 49735 as `[_49][7][3][5]`. Across the full range, only 2.2% of Telugu and 2.4% of Nepali three-digit numbers are single tokens. Since heights in the dataset are three-digit, a height comparison asks the model to recover magnitude from fragment sequences with no shared positional structure. `transitive` avoids this entirely, which is why it reaches 99.60% while `pairwise` reaches 46.00% in the same model – even though transitive chaining is arguably the harder inference. The gap is not about reasoning depth but about whether the quantities survive tokenization.

The two languages fail differently. Every logged Telugu `pairwise` error names an entity that *is* in the prompt: the model identifies the two candidates correctly and picks the wrong one. Given "మిక్సీ యొక్క బరువు 80 కిలోలు. కుర్చీ బరువు 51 కిలోలు." (mixer 80 kg, chair 51 kg, which is heavier?) it answers కుర్చీ (chair). That is a clean polarity error – the copy mechanism works, the magnitude comparison does not.

Nepali shows that error too, but adds two modes Telugu does not. The first is emitting a name that never appeared in the prompt – answering गणेश to a question about गोविन्द and विमला. Both hallucinated names are in the generator's entity pool but not in that prompt, so the model is retrieving a plausible name from the finetuning distribution instead of copying from context. The second is a boundary error: the genitive postposition fuses onto the answer (हेडफोनको where the reference is हेडफोन) or the name is truncated (किताब for किताबघर). Under strict exact match these score zero even though the right entity was identified. Devanagari writes postpositions attached to the noun, so the answer boundary is genuinely more ambiguous than in Telugu. Part of Nepali's deficit is therefore a scoring artifact rather than a reasoning deficit, and the true gap is likely smaller than 12 points.

The held-out pattern generalized. `transitive_4hop` retains most of the two-hop accuracy (81.95% and 68.67%) despite the training set containing no chain longer than two statements. If the transitive result were template memorization this should have collapsed. Both models learned the relation rather than the sentence shape; the modest drop is consistent with error compounding over an extra inference step.

3.2 Model H against Model L

Telugu finishes at 64.69% and Nepali at 52.74%, leading on every template. This reverses the Phase 2 ordering, where Nepali beat Telugu on every language-modeling metric (test perplexity 33.09 vs. 38.70; BPB 0.4256 vs. 0.4827).

The reversal is not contradictory, because the two phases measure different things. Perplexity rewards assigning high probability to the text distribution the model was trained on, and Nepali's advantage there is plausibly a property of its corpus rather than of the model: its manual tier is

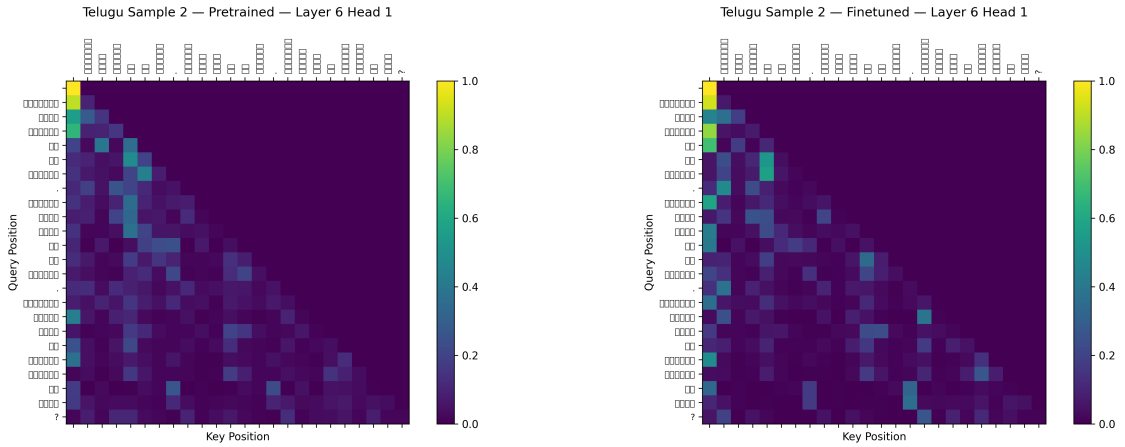
dominated by formulaic news and government text, and a more predictable corpus yields lower perplexity without implying a more capable model. Telugu's corpus is more heterogeneous – a much larger scraped news share across many more sites, plus more OCR material – which is harder to model but appears to have produced representations that transfer better to a task requiring entity tracking across clauses. Nepali won the metric that rewards predictability; Telugu won the one that rewards flexibility.

4 Post-Finetune Attention Analysis

I reran the Phase 2 `AttentionAnalyzer` on the finetuned checkpoints, comparing Layer 1 (early) and Layer 6 (late) against the pretrained checkpoint on comparative-reasoning prompts in each model's own language. Statistics are averaged over all heads in the layer and over two prompts per language (one pairwise, one transitive).

Table 5: Attention statistics before and after reasoning finetuning

	Telugu (H)			Nepali (L)		
Metric	Pretr.	Finet.	Δ	Pretr.	Finet.	Δ
Entropy, Layer 1	1.839	1.845	+0.3%	1.992	2.001	+0.5%
Entropy, Layer 6	1.256	1.277	+1.7%	1.369	1.357	−0.9%
Mean distance, Layer 1	6.007	5.984	−0.4%	7.140	7.091	−0.7%
Mean distance, Layer 6	6.479	6.063	**−6.4%**	8.041	7.467	**−7.1%**



(a) Pretrained (b) Finetuned

Figure 3: Telugu, Layer 6 Head 1, on the transitive prompt "పుస్తకం కంటే సైకిల్ ధరలో ఎక్కువ. సైకిల్ కంటే కారు ధరలో ఎక్కువ. పుస్తకం మరియు కారు లో ఎవరికి ఎక్కువ ధర ఉంది?". The finetuned map concentrates more attention mass nearer the diagonal – the visual form of the −6.4% mean-distance change in Table 5.

The change is strikingly localized. Every Layer 1 statistic moves by under 1% in both languages, which is within what a low-learning-rate finetune would perturb incidentally. The adaptation is

in the late layers, and specifically in mean attention distance, which falls 6.4% (Telugu) and 7.1% (Nepali) while entropy stays nearly static.

Those two facts together say more than either alone. Attention did not become sharper or more diffuse – steady entropy means the *concentration* of attention mass is unchanged. What changed is *where* that mass sits: the same amount of focus, moved closer to the query. On a comparative-reasoning prompt the decisive tokens are the entity names and comparison word in the immediately preceding clause, so pulling late-layer attention inward is a sensible adaptation.

That early layers are untouched fits the standard picture of finetuning. Early layers handle script-level and morphological processing common to all text in the language, and the reasoning task gives them no reason to change; late layers assemble the representation feeding the output head, which is exactly what must change when the task shifts from "continue this article" to "identify which entity satisfies the comparison". Both languages show the same pattern at similar magnitude despite different scripts, suggesting this is a property of the objective rather than of either language. One caveat: two prompts per language establishes the direction but not a precise magnitude.

5 Conclusion

Phase 3 is complete for both languages. Each model was finetuned independently from its own Phase 2 checkpoint on its own synthetic reasoning corpus, with disjoint entity pools and a test-only relation pattern for leakage control. Both went from 0.00% to a substantial exact-match score – 64.69% for Model H and 52.74% for Model L – while holding general-language degradation to +6.0% and +5.1%, after a first configuration that reached +264% before I started measuring it. Curriculum rebalancing moved Telugu `pairwise` accuracy from a 4–12% band to 46.00%, and held-out three-hop chains retained most of the two-hop accuracy, indicating the transitive result is relational rather than memorized. The attention analysis shows the adaptation is confined almost entirely to late layers, taking the form of shorter attention distance at constant entropy.

The clearest limitation is numeric: BPE destroys place-value structure, which caps both models on the comparison templates. The change I would make next is to space out digits in the reasoning data, writing 146 as `1 4 6` so segmentation is per-digit and magnitude is recoverable from token position. Since it should lift the numeric templates while leaving the transitive ones untouched, it would also serve as a clean test of the tokenization explanation above.

Every number and figure in this report comes from committed result files and real generations in the repository.